TEAM SEAT 03 OF 04 · INSTITUTIONAL SYSTEMS & DEVOPS ENGINEERING MANUAL

SOFTWARE DEVELOPER & SYSTEMS ARCHITECTURE MANUAL

Role Mandate: Hybrid Split-Compute, Parquet I/O, Discord Cogs, CI/CD & Cloud Uptime System: trading-sandbox v2.4

YOUR MISSION INSIDE THE 4-PERSON QUANTITATIVE TEAM

As the **Software & Systems Engineer (Seat 03)**, you own the reliability, decoupling, and 24/7 uptime of the platform. You enforce strict separation between the **Local Windows Research Desk** (used by Seat 02 for heavy grid searches and ML training) and the **Google Cloud e2-micro Ubuntu Execution Node** (which routes orders via Alpaca and streams telemetry to Seats 01 and 04 via Discord Cogs and the Web Dashboard).

3.1 HYBRID SPLIT-COMPUTE TOPOLOGY & DATA FLOW ARCHITECTURE

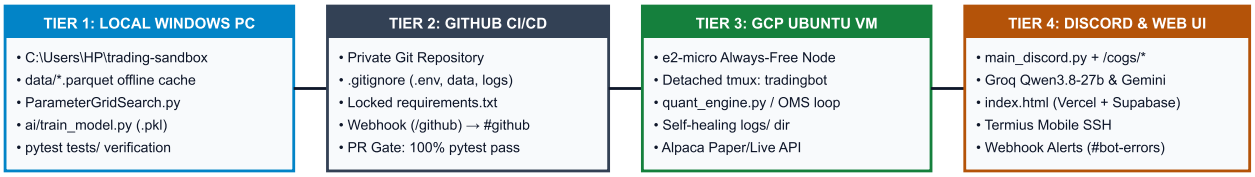


Figure 3.1: Production 4-Tier Split-Compute Architecture maintained by Seat 03 (Software & Systems Engineer).

3.2 MODULE-BY-MODULE INTERFACE CONTRACTS

File / Directory	Class / Entrypoint	Strict Engineering Contract & Bug-Prevention Rules
`data_fetcher.py` & `data/*.parquet`	`get_historical_bars()` `DataLayer.get_bars()`	Must query `StockHistoricalDataClient` with `feed=DataFeed.IEX` and `end_time = datetime.now(timezone.utc) - timedelta(minutes=15)` to prevent free-tier 401/empty DataFrame errors. Flattens MultiIndex via `df.xs(symbol, level="symbol")` and caches to `{symbol}_{tf}_{days}d.parquet`.
`quant_engine.py`	`QuantConfig`, `RiskEngine`, `ExecutionOMS`	Must initialize `DATA_DIR.mkdir(parents=True, exist_ok=True)` and `LOG_DIR.mkdir(parents=True, exist_ok=True)` *before* `logging.basicConfig()` to prevent Linux `FileNotFoundError` crashes on fresh cloud VMs.
`strategies/*.py`	`BollingerMeanReversion`	Exposes pure method `generate_signals(df: pd.DataFrame) -> pd.DataFrame` returning forward-filled `signal` column `{-1, 0, 1}`. Zero broker or network calls allowed inside strategy classes.
`main_discord.py`	`QuantPlatform(Bot)`	Intentionally lightweight engine block. Dynamically iterates `os.listdir('./cogs')` in `setup_hook()`, calls `load_extension()`, and syncs slash commands via `await self.tree.sync()`.
`cogs/research.py`	`ResearchCog` (`!arxiv`)	Queries `export.arxiv.org/api/query?search_query=cat:q-fin.TR+OR+cat:q-fin.ST&sortBy=submittedDate&sortOrder=descending` (note: `descending` spelled in full to prevent XML error responses) and summarizes via Groq `qwen/qwen3.8-27b`.

3.3 AUTOMATED QA GATE: PYTEST SUITE (TESTS/TEST_RISK_MANAGER.PY)

Before merging any pull request or deploying to the Google Cloud VM, Seat 03 must verify that all four risk circuit-breaker unit tests pass (`python -m pytest tests/`):

```
# VERIFICATION SUITE: tests/test_risk_manager.py
def test_daily_circuit_breaker(risk_engine):
    # 4.7% intraday drop ($105k -> $100k) exceeds 3% MAX_DAILY_DRAWDOWN -> Must return False
(Halt)
    assert risk_engine.check_daily_circuit_breaker(start_of_day_equity=105000.0) == False
    assert risk_engine.check_daily_circuit_breaker(start_of_day_equity=101000.0) == True

def test_position_sizing_by_risk_and_notional(risk_engine):
    # 1% risk rule ($100 entry, $90 stop -> 100 shares) vs 5% notional concentration cap ($99 stop
-> 50 shares)
    assert risk_engine.size_position(entry_price=100.0, stop_loss_price=90.0) == 100
    assert risk_engine.size_position(entry_price=100.0, stop_loss_price=99.0) == 50
    assert risk_engine.size_position(entry_price=100.0, stop_loss_price=100.0) == 0  # Zero-
division guard
```

3.4 CLOUD DEPLOYMENT, TMUX DAEMON & TROUBLESHOOTING RUNBOOK

```
# STEP 1 — LOCAL WINDOWS DESK: Run tests, freeze dependencies, and push to GitHub
(Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (&
.\venv\Scripts\Activate.ps1)
python -m pytest tests/
pip freeze > requirements.txt
git add . && git commit -m "feat: verified strategy and risk build" && git push origin main

# STEP 2 — GOOGLE CLOUD VM (Ubuntu LTS): Sync repo and manage 24/7 detached tmux daemon
ssh isyrarif@tradingplattform
cd ~/trading-sandbox && source venv/bin/activate
git pull origin main && pip install -r requirements.txt
tmux new -s tradingbot -d "python quant_engine.py"   # Or attach: tmux attach -t tradingbot
(Detach: Ctrl+B, D)
tail -f ~/trading-sandbox/logs/trading_bot.log       # Stream live execution telemetry
```

3.5 CROSS-TEAM HANDOFF PROTOCOL: HOW SEAT 03 WORKS WITH SEATS 01, 02 & 04

Team Counterpart	Engineering Input Received	Your Engineering Output & SLA Guarantee
Seat 01: Finance & Risk Officer	Updated risk thresholds (`.env`) and audit column requirements for Web UI Tab 3.	100% `pytest` enforcement on `RiskEngine`, real-time webhook embeds to `#risk`, and accurate CSV export formatting in `index.html`.
Seat 02: Quant Researcher	New strategy classes in `strategies/` and newly trained `models/rf_meta_model.pkl` files.	Fast local Parquet caching (`data_fetcher.py`), hot-reloading of `.pkl` models into `main_discord.py`, and cloud daemon deployment.
Seat 04: General Operations	Bug reports in `#bugs` or emergency `!cancelall` invocations in `#admin`.	99.9% uptime on the Vercel Web Dashboard (`index.html`), Discord slash command responsiveness, and `#bot-errors` stack-trace routing.

COPYRIGHT & SOFTWARE LICENSE NOTICE

Copyright © 2026 [Author / System Owner]. All Rights Reserved. Proprietary software engineering manual for Seat 03 of the `trading-sandbox` platform. Source code, cloud automation scripts, and Discord multi-agent cogs are strictly confidential.